

scheme in the data to prevent any overfitting cases. Lastly but not the least, convert any non-numeric datatypes into a format that the system can understand by using a hot encoding. This creates a separate column for all the categories in the dataset.

Once the model has run successfully, the next definite step should be to tune and refine parameters. Different models have different tuning parameters which the system can determine to zero in on the one that gives the best accuracy. Logistic regression, for example, uses an l1,l2 regularisation scheme while random forests use a gradient boosting technique to find optimal coefficients. Specify the number of hidden layers and the weight for the inputs in the code between the curly braces.

But don't just stop at the model accuracies to get the best API model. Test out the model with validation methods. This could be something as simple as a graph drawn to predict actual and expected outputs or something robust like the ROC curves. The models, of course, would grow more complicated when dealing with text, visual and audio data which is why Convolutional neural networks should be looked into.

The code in the next section offers a small glimpse into the basic structural skeleton for a functional API system in Python. The same can be used in TensorFlow and Pandas as well. Notice the use of the Dense function to specify how big the training and testing sets are going to be.

SAMPLE CODE FOR TESTS

```
• from keras.layers import Input, Dense
• from keras.models import Model
• inputs = Input(shape=(784,))
• x = Dense(64, activation='relu')(inputs)
• x = Dense(64, activation='relu')(x)
• predictions = Dense(10, activation='softmax')(x)
• model = Model(inputs=inputs, outputs=predictions)
• model.compile(optimizer='rmsprop',
•               loss='categorical_crossentropy',
•               metrics=['accuracy'])
• model.fit(data, labels) # starts training
• # Generate indexes of test and train
• idx_list = np.linspace(0,999,num=1000)
• idx_test = np.random.choice(n_row, size = 200,
replace=False)
• idx_train = np.delete(idx_list, idx_test).astype('int')
```